University of Idaho
Department of Computer Science
Coeur d'Alene



Lecture #5

Affine Transformations

Garrett Wells
revised August 27, 2024



Table of Contents I

Table of Contents

Review

Affine Transformations
- Translation
- Rotation
- Shear
- Scaling
  - Interpolation Strategies for Scaling
- Affine Transformations Generalized

Other Forms of Transformations

Table of Contents I

Table of Contents

Review

Affine Transformations
Translation
Rotation
Shear
Scaling
Interpolation Strategies for Scaling
Affine Transformations Generalized

Other Forms of Transformations

Review

- OpenCV camera calibration

Review
- OpenCV camera calibration

Pixel Access in Images

Image

Represented by a series of 2D matrices. Each index represents a value from a physical pixel.

- each pixel is defined by multiple values
- matrices represent colors, depth, or other per pixel values

Pixel Access in Images
Image
Represented by a series of 2D matrices. Each index represents a value from a physical pixel.
- each pixel is defined by multiple values
- matrices represent colors, depth, or other per pixel values

Affine Transformations

Wikipedia: Affine Transformation

“In Euclidean geometry, an affine transformation or affinity (from the Latin, affinis, "connected with") is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.” [1]

Affine Transformations
Wikipedia: Affine Transformation
“In Euclidean geometry, an affine transformation or affinity (from the Latin, affinis, "connected with") is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.” [1]

Affine Transformations

OpenCV: Affine Transformation

Defined as a transformation that can be expressed through **matrix multiplication** (*linear transformation*) followed by **vector addition** (*translation*) [2].

Affine Transformations
OpenCV: Affine Transformation
Defined as a transformation that can be expressed through **matrix multiplication** (*linear transformation*) followed by **vector addition** (*translation*) [2].

Affine Transformations

- Translation: changing pixel location(x/y)
- Rotation: changing pixel location by θ
- Shear: displace pixels proportional to distance from parallel line
- Scaling: image size up/down

Affine Transformations
- Translation: changing pixel location(x/y)
- Rotation: changing pixel location by θ
- Shear: displace pixels proportional to distance from parallel line
- Scaling: image size up/down

Affine Transformations

How do affine transformations work?

- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix

Affine Transformations
How do affine transformations work?
- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix

1. Reminder, *homogeneous coordinates* have scale factor W such that $(X, Y) \rightarrow (XW, YW, W)$.

Affine Transformations

How do affine transformations work?

- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:

$$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}_{2x2} \quad B = \begin{bmatrix} b_{00} \\ b_{10} \end{bmatrix}_{2x1}$$

Affine Transformations
How do affine transformations work?
- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:
$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}_{2x2} \quad B = \begin{bmatrix} b_{00} \\ b_{10} \end{bmatrix}_{2x1}$

1. Reminder, *homogeneous coordinates* have scale factor W such that $(X, Y) \rightarrow (XW, YW, W)$.

Affine Transformations

How do affine transformations work?

- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:

$$M = [A|B] = \begin{bmatrix} a_{00} & a_{01} & b_{00} \\ a_{10} & a_{11} & b_{10} \end{bmatrix}_{2x3}$$

1. Reminder, *homogeneous coordinates* have scale factor W such that $(X, Y) \rightarrow (XW, YW, W)$.

Affine Transformations

How do affine transformations work?

- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:

$$T = A \cdot \begin{bmatrix} x \\ y \end{bmatrix} + B$$

OR

$$T = M \cdot \begin{bmatrix} x & y & 1 \end{bmatrix}^T$$

1. Reminder, *homogeneous coordinates* have scale factor W such that $(X, Y) \rightarrow (XW, YW, W)$.

Affine Transformations

How do affine transformations work?

- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:

$$T = \begin{bmatrix} a_{00}x + a_{01}y + b_{00} \\ a_{10}x + a_{11}y + b_{10} \end{bmatrix}$$

Affine Transformations
How do affine transformations work?
- Multiply 2×2 matrix, then add 2×1 matrix
- Or use homogeneous coordinates and multiply by 2×3 matrix
- According to [2], the following is how Affine Transformations are represented:
$T = \begin{bmatrix} a_{00}x + a_{01}y + b_{00} \\ a_{10}x + a_{11}y + b_{10} \end{bmatrix}$

1. Reminder, *homogeneous coordinates* have scale factor W such that $(X, Y) \rightarrow (XW, YW, W)$.

Affine Transformations In OpenCV

```
# Apply affine transformation defined by M to image (src)
img = cv.warpAffine(src,
                    M,
                    dsize=(columns, rows),
                    flags=INTER_LINEAR,
                    borderMode=BORDER_CONSTANT,
                    borderValue=Scalar())
```


Affine Transformations In OpenCV

```
# Apply affine transformation defined by M to image (src)
img = cv.warpAffine(src,
                    M,
                    dsize=(columns, rows),
                    flags=INTER_LINEAR,
                    borderMode=BORDER_CONSTANT,
                    borderValue=Scalar())
```

1. To perform an affine transformation, we need to create *M* and use `warpAffine(..., M, ...)`
2. OpenCV provides various functions for calculating *M*, such as `getAffineTransform(src, dst)`, `getRotationMatrix2D(center, angle, scale)`

Affine Transformations in OpenCV

```
# Get 2x3 M from 2 sets of 3 points (x, y)
cv.getAffineTransform(
                        src,
                        dst
                    )
# Get 2x3 rotation matrix
cv.getRotationMatrix2D(
                        center,
                        angle,
                        scale
                    )
```


Affine Transformations in OpenCV

```
# Get 2x3 M from 2 sets of 3 points (x, y)
cv.getAffineTransform(
                        src,
                        dst
                    )
# Get 2x3 rotation matrix
cv.getRotationMatrix2D(
                        center,
                        angle,
                        scale
                    )
```

Translation

- Moving the image around within the frame
- Translating pixels from one location to another

Translation
- Moving the image around within the frame
- Translating pixels from one location to another

Translation

Cont'd

- Since we are moving in a straight line ...
- we only need the B matrix
- Thus M is...

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

Translation
Cont'd
- Since we are moving in a straight line ...
- we only need the B matrix
- Thus M is...
$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$

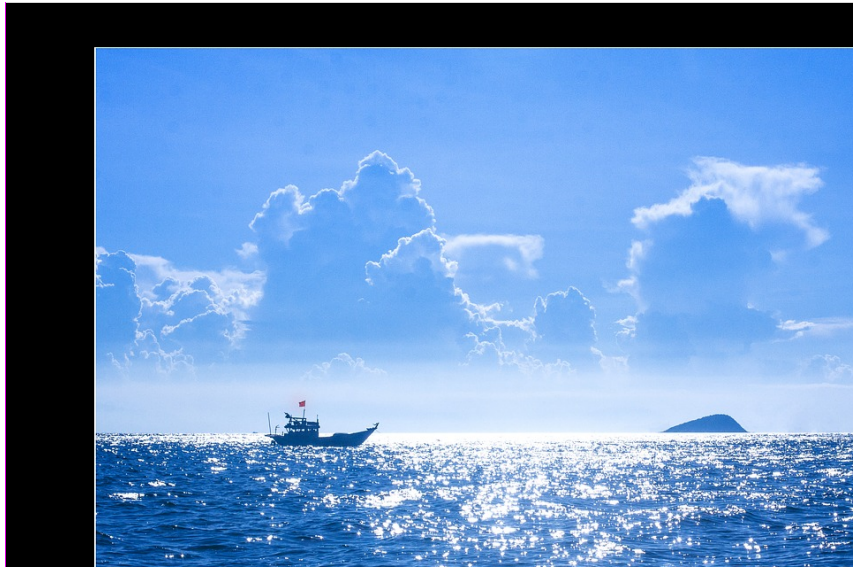
Translation Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# x -> 100, y -> 50
M = np.float32([
                [1, 0, 100],
                [0, 1, 50 ]
            ])
img = cv.warpAffine(img, M, (columns, rows))
mv_library.showImage(img)
```


Translation Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# x -> 100, y -> 50
M = np.float32([
                [1, 0, 100],
                [0, 1, 50 ]
            ])
img = cv.warpAffine(img, M, (columns, rows))
mv_library.showImage(img)
```

Output



Output



Rotation

- Rotate the image by angle θ

Rotation
- Rotate the image by angle θ

Rotation

Cont'd

- Here, we rely on A matrix
- For a pure rotation, B (translation) isn't needed.

$$A = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Rotation
Cont'd
- Here, we rely on A matrix
- For a pure rotation, B (translation) isn't needed.
$A = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

1. This should look familiar from the rotation matrix used for the camera extrinsic matrix.

Rotation Code Example

```python
# Rotate image from center
center = ((columns-1)/2.0, (rows-1)/2.0)
angle = 90
scale = 1
M = cv.getRotationMatrix2D(center, angle, scale)
img = cv.warpAffine(img, M, (columns, rows))
```


Rotation Code Example

Rotate image from center
center = ((columns-1)/2.0, (rows-1)/2.0)
angle = 90
scale = 1
M = cv.getRotationMatrix2D(center, angle, scale)
img = cv.warpAffine(img, M, (columns, rows))

Output

Output

Shear

$$M = [A|B] = \begin{bmatrix} 1 & sh_y & 0 \\ sh_x & 1 & 0 \end{bmatrix}$$

Wikipedia: Shear

Shear
$M = [A|B] = \begin{bmatrix} 1 & sh_y & 0 \\ sh_x & 1 & 0 \end{bmatrix}$
Wikipedia: Shear

Shear Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# All pixels farther to right as y increases
#   note: (0,0) is top left of image frame
M = np.float32([
                [1, 0.5, 0],
                [0, 1, 0 ]
            ])
img = cv.warpAffine(img, M, (columns, rows))
```


Shear Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# All pixels farther to right as y increases
#   note: (0,0) is top left of image frame
M = np.float32([
                [1, 0.5, 0],
                [0, 1, 0 ]
            ])
img = cv.warpAffine(img, M, (columns, rows))
```

Output



Output



Scaling

- There are several different methods
  1. `cv.resize()`, using geometric transformation
  2. Image Pyramids

Scaling
- There are several different methods
  1. `cv.resize()`, using geometric transformation
  2. Image Pyramids

Scaling

Cont'd

$$M = [A|B] = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix}$$

Scaling
Cont'd
$M = [A|B] = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix}$

1. Notice that this is taking the same form as the camera intrinsic matrix with focal length(f_x,f_y), replaced by a different scale factor.
2. An *increase* in focal length means that an object in the original image takes up *more* of the frame. The difference is that here we aren't gaining information by scaling up, we are having to replicate or guess what values to fill space with.

Scaling

Cont'd

1. option 1, keep original values
2. option 2, interpolate what the new values should be

Scaling
Cont'd
1. option 1, keep original values
2. option 2, interpolate what the new values should be

Scaling
Cont'd

Option 1 Keep original values

$$\begin{bmatrix} 2 & 2 & 2 & 2 \\ 6 & 6 & 6 & 6 \\ 2 & 2 & 2 & 2 \\ 6 & 6 & 6 & 6 \end{bmatrix}$$

▼

$$\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$$

Scaling
Cont'd
Option 1 Keep original values

Scaling

Cont'd

Option 2 Interpolate what the new values should be

$$\begin{bmatrix} 2 & 2 & 2 & 2 \\ 6 & 6 & 6 & 6 \\ 2 & 2 & 2 & 2 \\ 6 & 6 & 6 & 6 \end{bmatrix}$$

▼

$$\begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

Scaling
Cont'd
Option 2 Interpolate what the new values should be
▼

Interpolation Strategies: Nearest Neighbors

New pixel values often interpolated using a strategy called “Nearest Neighbors” [3].

Interpolation Strategies: Nearest Neighbors

New pixel values often interpolated using a strategy called “Nearest Neighbors” [3].

Interpolation Strategies: Nearest Neighbors

New pixel values often interpolated using a strategy called “Nearest Neighbors” [3].



Interpolation Strategies: Nearest Neighbors
New pixel values often interpolated using a strategy called “Nearest Neighbors” [3].



Interpolation Strategies: Linear Interpolation

Weighted average of 4 pixels *beside* the target pixel [4].

Interpolation Strategies: Linear Interpolation
Weighted average of 4 pixels *beside* the target pixel [4].

Interpolation Strategies: Linear Interpolation

Weighted average of 4 pixels *beside* the target pixel [4].



Interpolation Strategies: Linear Interpolation
Weighted average of 4 pixels *beside* the target pixel [4].



Interpolation Strategies: Cubic

Uses weighted average of 16 neighboring pixels (4×4) [5].

Interpolation Strategies: Cubic
Uses weighted average of 16 neighboring pixels (4×4) [5].

Interpolation Strategies: Cubic

Uses weighted average of 16 neighboring pixels (4×4) [5].



Interpolation Strategies: Cubic
Uses weighted average of 16 neighboring pixels (4×4) [5].



Interpolation Strategies: Area

- Average a “box” of pixels [6]
- Usually used for down scaling

Interpolation Strategies: Area
- Average a “box” of pixels [6]
- Usually used for down scaling

Interpolation Strategies: Area

- Average a “box” of pixels [6]
- Usually used for down scaling



Interpolation Strategies: Area
- Average a “box” of pixels [6]
- Usually used for down scaling



Interpolation Strategies: Others

Other algorithms do exist, but we won't examine them now.

Interpolation Strategies: Others
Other algorithms do exist, but we won't examine them now.

Affine Transformations Generalized

- OpenCV can generate any affine transformation you desire
- `cv.getAffineTransform()`
- three points from input image → position in output image

Affine Transformations Generalized
- OpenCV can generate any affine transformation you desire
- cv.getAffineTransform()
- three points from input image → position in output image

General Affine Transformation

```
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
first_pts = np.float32([[50,50],[200, 50],[50,200]])
next_pts = np.float32([[10,100],[200, 50],[100,250]])
M = cv.getAffineTransform(first_pts, next_pts)
img = cv.warpAffine(img, M, (columns, rows))
```


General Affine Transformation

```
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
first_pts = np.float32([[50,50],[200, 50],[50,200]])
next_pts = np.float32([[10,100],[200, 50],[100,250]])
M = cv.getAffineTransform(first_pts, next_pts)
img = cv.warpAffine(img, M, (columns, rows))
```

Output



Output



Perspective & Value Transformations

1. Perspective Transform
2. Color Channels
3. Brightness/Contrast
4. Gamma Correction

Perspective & Value Transformations

1. Perspective Transform
2. Color Channels
3. Brightness/Contrast
4. Gamma Correction

Perspective Transform

- Not an affine transformation
- Doesn’t preserve parallel lines
- Warps the image’s perspective
- `cv.getPerspectiveTransform()` takes 4 input points and where they should be in the output

Perspective Transform
- Not an affine transformation
- Doesn't preserve parallel lines
- Warps the image's perspective
- cv.getPerspectiveTransform() takes 4 input points and where they should be in the output

Warp Perspective Code Example

```
rows, columns, channels = img.shape
first_pts = np.float32([[50,50],[400, 50],[50,400], [400, 400]])
next_pts = np.float32([[50, 0],[600, 0],[50, 50], [600, 600]])

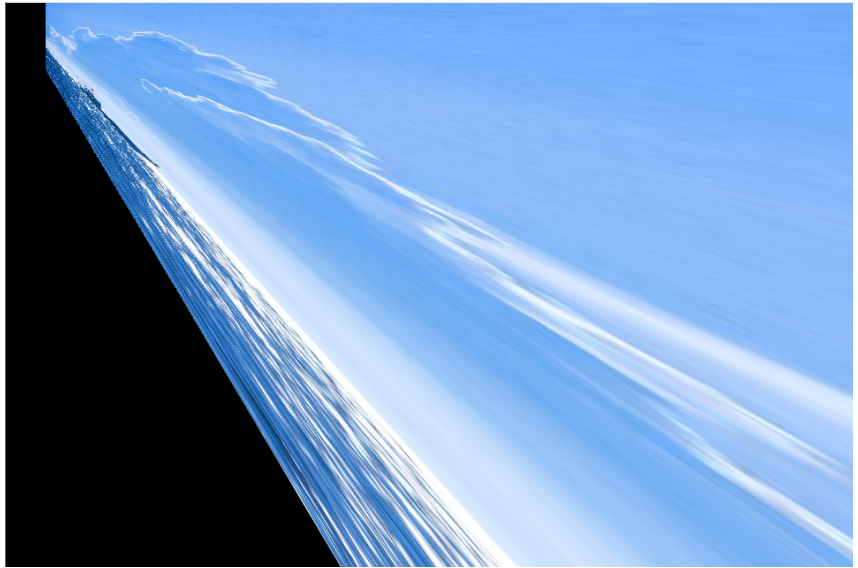
M = cv.getPerspectiveTransform(first_pts, next_pts)
img = cv.warpPerspective(img, M, (columns, rows))
```


Warp Perspective Code Example

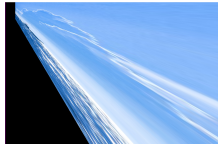
rows, columns, channels = img.shape
first_pts = np.float32([[50,50],[400, 50],[50,400], [400, 400]])
next_pts = np.float32([[50, 0],[600, 0],[50, 50], [600, 600]])

M = cv.getPerspectiveTransform(first_pts, next_pts)
img = cv.warpPerspective(img, M, (columns, rows))

Output



Output



Color Channels

- Color channels may be split

Color Channels
- Color channels may be split

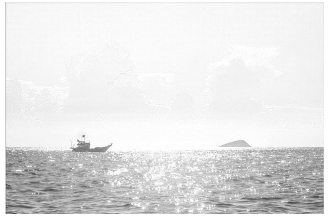
Color Channels Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# splits into blue, green, red channels
# can be passed to imshow(...)
b, g, r = cv.split(img)
# merge into one image
img = cv.merge((b, g, r))
```


Color Channels Code Example

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# splits into blue, green, red channels
# can be passed to imshow(...)
b, g, r = cv.split(img)
# merge into one image
img = cv.merge((b, g, r))
```

Output





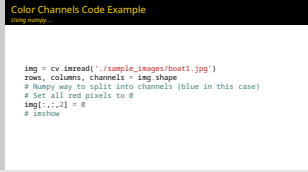
Output



Color Channels Code Example

Using numpy…

```python
img = cv.imread('./sample_images/boat1.jpg')
rows, columns, channels = img.shape
# Numpy way to split into channels (blue in this case)
# Set all red pixels to 0
img[:,:,2] = 0
# imshow
```

Output



Output



Brightness & Contrast

- Modify pixel intensity in image
- Linear transformation

$$g = af + b$$

g	the new pixel value
f	old pixel value
a	alpha value (contrast & gain)
b	beta value (brightness & bias)

Brightness & Contrast
- Modify pixel intensity in image
- Linear transformation

$g = af + b$

g the new pixel value
f old pixel value
a alpha value (contrast & gain)
b beta value (brightness & bias)

Brightness & Contrast Code Example

```python
img = cv.imread('./sample_images/octo1.jpg')
alpha = 2.2
beta = 20.0
img = cv.convertScaleAbs(img, alpha=alpha, beta=beta)
```


Brightness & Contrast Code Example

```python
img = cv.imread('./sample_images/octo1.jpg')
alpha = 2.2
beta = 20.0
img = cv.convertScaleAbs(img, alpha=alpha, beta=beta)
```

Output



Output



Gamma Correction

- Similar to brightness/contrast
- Non-linear function

$$O = (\frac{I}{255})^{\gamma} \cdot 255$$

O	output value
I	input value
γ	gamma

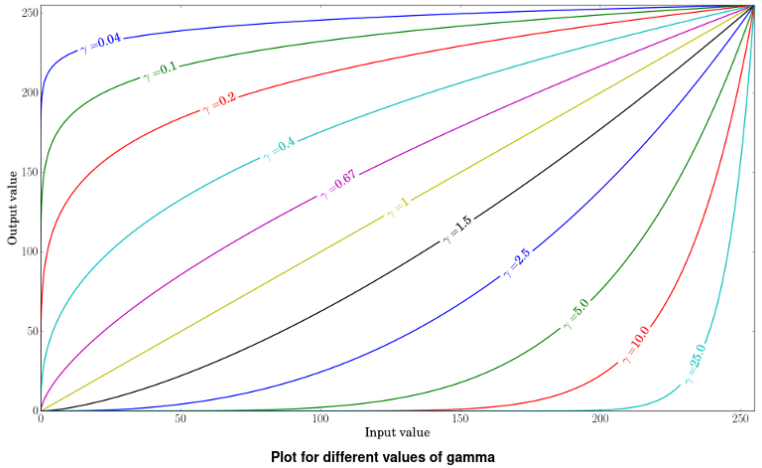
1. When $\gamma < 1$, dark areas will be brighter, histogram shifts right. When $\gamma > 1$, darkens image, shifts histogram left.
2. Non-linear nature limits how many bright or dark pixel values get clipped. That means if brightening image, pixels close to 255 value won't be brightened as much as pixels at 25.

Gamma Correction
- Similar to brightness/contrast
- Non-linear function
$O = (\frac{I}{255})^{\gamma} \cdot 255$
O output value
I input value
γ gamma

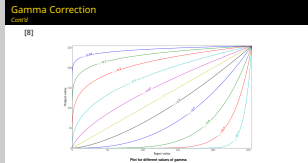
Gamma Correction

Cont'd

[8]



Plot for different values of gamma



Gamma Correction Code Example

```python
gamma = 3.0
look_up = np.empty((1,256), np.uint8)
for i in range(256):
    look_up[0,i] = np.clip(
                           pow(i / 255.0, gamma) * 255.0,
                           0,
                           255)
img = cv.LUT(img, look_up)
```


Gamma Correction Code Example

```python
gamma = 3.0
look_up = np.empty((1,256), np.uint8)
for i in range(256):
    look_up[0,i] = np.clip(
                           pow(i / 255.0, gamma) * 255.0,
                           0,
                           255)
img = cv.LUT(img, look_up)
```

Output



Output



Bibliography I

[1] *Affine transformation*, in *Wikipedia*, Aug. 24, 2024. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Affine_transformation&oldid=1242090128 (visited on 08/26/2024).

[2] "OpenCV: Affine Transformations," (), [Online]. Available: https://docs.opencv.org/4.x/d4/d61/tutorial_warp_affine.html (visited on 08/25/2024).

[3] K. bibinitperiod Atul, "Image Processing – Nearest Neighbour Interpolation," TheAILearner. (Dec. 29, 2018), [Online]. Available: https://theailearner.com/2018/12/29/image-processing-nearest-neighbour-interpolation/ (visited on 08/25/2024).

Bibliography I

[1] *Affine transformation*, in *Wikipedia*, Aug. 24, 2024. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Affine_transformation&oldid=1242090128 (visited on 08/26/2024).

[2] "OpenCV: Affine Transformations," (), [Online]. Available: https://docs.opencv.org/4.x/d4/d61/tutorial_warp_affine.html (visited on 08/25/2024).

[3] K. bibinitperiod Atul, "Image Processing – Nearest Neighbour Interpolation," TheAILearner. (Dec. 29, 2018), [Online]. Available: https://theailearner.com/2018/12/29/image-processing-nearest-neighbour-interpolation/ (visited on 08/25/2024).

Bibliography II

[4] K.
bibinitperiod Atul, “Image Processing – Bilinear Interpolation,”
TheAILearner. (Dec. 29, 2018), [Online]. Available:
https://theailearner.com/2018/12/29/image-processing-bilinear-interpolation/ (visited on 08/25/2024).

[5] K.
bibinitperiod Atul, “Image Processing – Bicubic Interpolation,”
TheAILearner. (Dec. 29, 2018), [Online]. Available:
https://theailearner.com/2018/12/29/image-processing-bicubic-interpolation/ (visited on 08/25/2024).

[6] W. Dong, “What is OpenCV’s INTER_AREA Actually Doing?” Medium. (Jun. 26, 2018), [Online]. Available:
https://medium.com/@wenrudong/what-is-opencvs-inter-area-actually-doing-282a626a09b3 (visited on 08/25/2024).

Bibliography II
[4] K. bibinitperiod Atul, “Image Processing – Bilinear Interpolation,” TheAILearner. (Dec. 29, 2018), [Online]. Available: https://theailearner.com/2018/12/29/image-processing-bilinear-interpolation/ (visited on 08/25/2024).
[5] K. bibinitperiod Atul, “Image Processing – Bicubic Interpolation,” TheAILearner. (Dec. 29, 2018), [Online]. Available: https://theailearner.com/2018/12/29/image-processing-bicubic-interpolation/ (visited on 08/25/2024).
[6] W. Dong, “What is OpenCV’s INTER_AREA Actually Doing?” Medium. (Jun. 26, 2018), [Online]. Available: https://medium.com/@wenrudong/what-is-opencvs-inter-area-actually-doing-282a626a09b3 (visited on 08/25/2024).

Bibliography III

[7] “OpenCV: Geometric Transformations of Images,” (), [Online]. Available: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html (visited on 08/25/2024).

[8] “OpenCV: Changing the contrast and brightness of an image!” (), [Online]. Available: https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html (visited on 08/25/2024).

Bibliography III

[7] “OpenCV: Geometric Transformations of Images,” (), [Online]. Available: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html (visited on 08/25/2024).

[8] “OpenCV: Changing the contrast and brightness of an image!” (), [Online]. Available: https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html (visited on 08/25/2024).